

O que é GitHub?

O GitHub é uma plataforma online que hospeda repositórios Git, ou seja, armazena e permite gerenciar projetos de software com controle de versão.

O que é Git?

É um sistema de controle de versão distribuído, um software de código aberto, usado para rastrear e gerenciar as mudanças em arquivos ao longo do tempo. Git permite que você controle e acompanhe o histórico de modificações em projetos, especialmente em desenvolvimento de software, permitindo que você volte a versões anteriores, compare alterações e trabalhe em equipe de forma colaborativa.

Não confunda!!! Git **não é a mesma coisa** que GitHub, GitHub trabalha com Git que é o software de controle de versão.

Seguindo para a parte prática, bem mais divertida... Imagine que você está desenvolvendo um projeto e agora seu amigo também quer participar do desenvolvimento. Podemos usar o drive e ter que mesclar os códigos manualmente, ou aguardar um terminar o que está fazendo para o outro trabalhar, estressante e perde muito tempo. Mas com o Git podemos fazer algo melhor, usando GitHub como plataforma vamos usar esse nosso amigo para desenvolver de forma colaborativa, como? Simples, o Git permite que duas ou mais pessoas desenvolvam o mesmo projeto sem que uma tenha que esperar a outra terminar, mas cuidado, não é tão simples assim. Por exemplo, duas pessoas não podem modificar o mesmo código ao mesmo tempo, imagine um código simples em python:

Modificação de Maria:

```
print("Hello World!!!")
```

Modificação de Pedro:

```
print("Olá Mundo!!!")
```

Quando eles enviarem, o Git vai olhar para as duas atualizações e não vai saber qual manter, então surgirá o famoso e temido Conflito de Merge! Mas calma, dá para consertar, no GitHub irá mostrar os arquivos e você deixa o código que quiser, ou até mesmo os dois, aí faz o merge.

Na escola temos 4 bimestres, posso criar uma branch para cada bimestre?

Sim, você pode, mas não recomendo. Um "branch" (ou "ramo" em português) é uma linha de desenvolvimento separada do código principal (geralmente chamado de main ou master). Imagine um ramo de uma árvore; ele permite que desenvolvedores trabalhem em novas funcionalidades, correções de bugs ou experimentações sem

afetar o código principal. O que eu recomendo é: crie um repositório para cada bimestre, por exemplo: 1Bim2025, 2Bim2025.

Mas eu quero um repositório que tenha todos os bimestres.

Alguns decidem dividir isso em pastas, exemplo:

repositórioAno2025

- pasta1Bim

 - codigo.py

- pasta2Bim

 - codigo.php

Eu aconselho trabalhar com subtrees. Imagine um repositório pai que tem repositórios filhos. Nesse caso o repositório pai seria o Ano2025 e os repositórios filhos seriam o 1Bim, 2Bim, etc. A estrutura fica assim:

Ano2025

- src

 - 1Bim

 - codigo.py

 - 2Bim

 - codigo.php

Mas o que muda da divisão em pastas?

A organização, você não vai commitar direto no repositório Ano2025, você vai commitar nos repositórios filhos, então só depois você irá mesclar isso no repositório pai.

Como faço isso?

CRIANDO SUBTREES:

Na pasta que estará o repositório pai:

```
git init
```

```
git remote add origin https://github.com/seuUsuario/repo-pai.git
```

```
git remote add repo-filho1 https://github.com/seuUsuario/repo-filho1.git
```

```
git fetch repo-filho1
```

```
git subtree add --prefix=nome/diretório repo-filho1 branch-filho1
```

```
git remote add repo-original2 https://github.com/seuUsuario/repo-original2.git
```

```
git fetch repo-filho2
```

```
git subtree add --prefix=nome/diretório repo-filho2 branch-filho2
```

```
git push -u origin branch-pai
```

PARA ATUALIZAR O REPOSITÓRIO:

```
git fetch repo-filho1
```

```
git subtree pull --prefix=nome/diretório repo-filho1 branch-filho1
```

```
git fetch repo-filho2
```

```
git subtree pull --prefix=nome/diretório repo-filho2 branch-filho2
```

```
git push
```

Como ficaria para o exemplo do ano?

CRIANDO SUBTREES:

Na pasta que estará o repositório pai:

```
git init
```

```
git remote add origin https://github.com/seuUsuario/Ano2025.git
```

```
git remote add Bim1 https://github.com/seuUsuario/Bim1.git
```

```
git fetch Bim1
```

```
git subtree add --prefix=src/Bim1 Bim1 main
```

```
git remote add Bim2 https://github.com/seuUsuario/Bim2.git
```

```
git fetch Bim2
```

```
git subtree add --prefix=src/Bim2 Bim2 main
```

```
git push -u origin main
```

PARA ATUALIZAR O REPOSITÓRIO:

git fetch Bim1

git subtree pull --prefix=src/Bim1 Bim1 main

git fetch Bim2

git subtree pull --prefix=src/Bim2 Bim2 main

git push

OBS: As branches que são citadas são as branches padrão, para onde o código principal vai.

Observe que Bim1 após o git remote add nada mais é do que o nome de uma variável que guarda o remote de um repositório, assim como Bim2 e origin. Só uma recomendação, crie com letras minúsculas e se for separado, use hífen na separação.

BONS COSTUMES

PRs

Sempre gere os famosos PRs (Pull Request). Basicamente é uma forma de manter o código principal longe de problemas e conflitos. Imagine que você está desenvolvendo, é o fim do expediente e você vai fazer um belo de um commit para ir dormir. Você manda o commit direto para o código principal (branch principal), no dia seguinte você acorda e vai testar e vê que algo aconteceu e tudo parou de funcionar, pois é, você enviou um código com bug. Se você tivesse gerado a PR isso não aconteceria.

Vamos voltar no dia anterior, vamos seguir os mesmos passos, mas desta vez você criou uma branch nova, agora fez um commit e criou um PR, pode dormir de consciência limpa. No dia seguinte você acorda e testa o código que está no PR mas que não foi mergeado. Bug, mas calma, seu código principal ainda funciona, é só consertar e mergear.

Conventional Commits

Pense, se criaram uma convenção é porque precisa, não é frescura ou burocracia do seu líder técnico, gestor, ou seja lá como se chama, é **NECESSÁRIO**.

Imagine que você está desenvolvendo um código e seu amigo decide que quer ajudar, mas aí ele vai trabalhar com algo grande, com muitas linhas de código.

Dúvida que você vai ler o código inteiro para tentar entender o que aquilo faz, você só quer ver se funciona. Pergunte para ele o que aquele código faz, legal ele não te

responde já tem dois dias e o PR está esperando para ser mergeado. Para isso existe a mensagem de commit...

Na mensagem de commit o seu amigo vai escrever o que foi feito, como funciona, para que serve, o que afeta, etc. Você vai ler a mensagem e vai saber o que aquele código faz. A partir daí você testa, verifica se está funcionando, se estiver, faz o merge, senão fecha o PR sem merge e manda ele consertar.